mmore: code deep dive, state of the art and proposed changes

Oct 10, 2026 · @Yassine

Summary

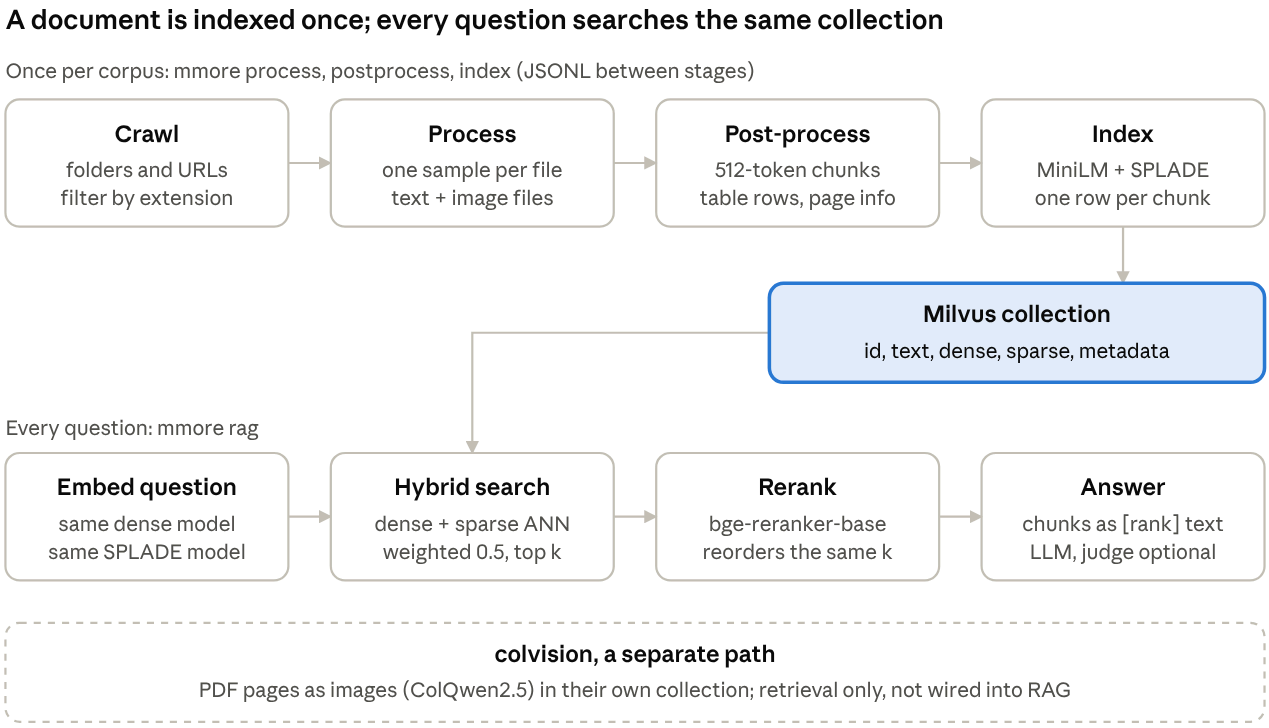
mmore is a five-stage pipeline: **process** files into text and images, **post-process** them into chunks, **index** chunks in Milvus with a dense and a sparse embedding, **retrieve** with a fused hybrid search plus a cross-encoder reranker, and **answer** with an LLM, optionally behind a corrective judge or a privacy layer. Every stage is a CLI command (`mmore process` , `postprocess` , `index` , `retrieve` , `rag`) reading a YAML config, and stages talk through JSONL files of `MultimodalSample` objects. A separate `colvision` path embeds PDF pages as images (ColQwen2.5) but stops at retrieval.

This review covers commit `1377ce0` (Oct 2026). Every claim below points to a file and line. Claims marked **verified** were reproduced by running code in the project's own environment; the rest come from reading the code.

What matters most for the thesis:

- **The default text pipeline is weaker than the paper describes.** Fast mode (the code default) reads the PDF text layer with PyMuPDF and does no OCR. Marker OCR runs only with `use_fast_processors: false` .
- **Provenance is lost in the standard CLI flow.** Chunk ids are salted hashes (verified), and page/paragraph positions vanish when `postprocess` reads the JSONL (verified). Only the live indexing API keeps file ids.
- **The reranker cannot recover missed documents.** It reorders the same k hits the hybrid search returned; it never sees a larger candidate pool.
- **The `dense` and `sparse` search modes are swapped** (verified). No pipeline code uses them yet, but a routing experiment would.
- **Page-image retrieval exists but is not part of RAG**, and it hands the generator the page's text layer, not the image.
- **The judge is CRAG-style with no abstain action**, and its web fallback (DuckDuckGo) cannot help with private documents.

The map



mmore pipeline · document path, query path, colvision

The top row runs once per corpus and hands JSONL files from stage to stage; the bottom row runs for every question against the same Milvus collection. The reranker only reorders the k hits the search returns, and colvision never reaches the answer step.

Component by component

Paths are relative to `src/mmore/`. Line numbers are at commit `1377ce0`.

0. The shared data type

`MultimodalSample` (`type.py:71-143`) carries `text`, a list of `modalities` (each `{type: "image", value: <png path>}`), `metadata` and an `id`. Images are not inlined: processors save them as PNG files and leave an `<attachment>` tag in the text where each one appeared. The `id` defaults to `str(hash(text))` (`type.py:90`), and `to_dict` (`:98-109`) writes neither `id` nor `document_id`, so ids are rebuilt on every load.

1. Process: files to text

1. **Crawl** (`process/crawler.py:244-273`). Each `data_path` entry must be a directory or a URL; a single file path raises `Invalid path`. Directories are walked recursively and filtered by an extension list (`run_process.py:114-137`).
2. **Route** (`process/dispatcher.py:194-223`). Each file goes to the first registered processor whose `accepts()` matches its extension, or to the one named in `processor_selection`. Processors are registered alphabetically by module (`process/processors/__init__.py:9-19`). Files are packed into batches of 100 by default (best-fit, `:391-432`).
3. **Extract.** One processor per format:
   - **PDF** (`process/processors/pdf_processor.py`). Fast mode (`process_fast` , `:264-332`) reads the embedded text layer with PyMuPDF, page by page, and runs `clean_text` on it. Normal mode (`process` , `:196-215`) runs marker (layout, OCR, tables) with paginated output, turns image links into `<attachment>`, and records `paragraph_starts` = (character offset, page, paragraph index) for every paragraph (`:218-262`). Several GPUs and 10+ files: one marker process per GPU (`:145-191`).
   - **DOCX** via mammoth to HTML to Markdown; **PPTX** shape text sorted top to bottom, plus notes; **XLSX/CSV** via pandas `to_string` ; **HTML/MD** to Markdown; **EML** headers + `text/plain` parts only; **TXT** as is; **audio/video** via Whisper (`large-v3-turbo` , or `tiny` in fast mode) plus one frame every 10 s; **URLs** via trafilatura.
   - `clean_text` (`process/utils.py:27-58`) is applied by most of them. **Verified:** it turns `\n\n` into `\n` , squeezes repeated spaces and replaces any email address with `email@email.com` .
4. **Write.** Each processor appends to `processors/<Name>/results.jsonl` ; the run then merges everything into `merged/merged_results.jsonl` (`run_process.py:57-84`). One sample = one whole file. `previous_results` lets a rerun reuse files whose modification time is older than their `processed_at` (`process/incremental.py:61-81`), without checking whether the config changed.

2. Post-process: documents to chunks

`PPPipeline` (`process/post_processor/pipeline.py`) runs a configured list of steps over every sample and writes `final.jsonl` (or the `.jsonl` path given).

- **Chunker** (`process/post_processor/chunker/multimodal.py`). Default: chonkie `SentenceChunker` , GPT-2 tokenizer, 512 tokens, no overlap (**verified**, chonkie 0.5.1). Markdown tables are detected first and, by default, split into one chunk per row with

the header repeated (`table_handling: single_row`). Each chunk keeps the parent's metadata, gets the images whose `<attachment>` tags fall inside it (`:67-84`), and gets `paragraph_positions` = the (page, paragraph) pairs it overlaps (`:248-273`). Chunk id = `<parent id>+<i>` (`:241`).

- **Other steps:** language/word/modality/file-name taggers; datatrove quality filters (Gopher, FineWeb, C4, language, regex); offline translation with Argos Translate; LLM-based entity/relation extraction (`ner`); `metafuse` , which pastes chosen metadata into the chunk text.

3. Index: chunks to Milvus

`Indexer` (`index/indexer.py`) creates one Milvus collection per corpus with fields `id` (primary key), `document_id` , `text` (max 65,535 characters), a dense vector and a sparse vector. Dynamic fields hold all metadata (`file_path` , `paragraph_positions` , and so on) (`:115-140`).

- **Dense:** any Hugging Face sentence-transformers model through LangChain (`rag/model/dense/base.py:69-84`), or OpenAI/Cohere/Mistral/Bedrock by name. No query/passage prefixes are added. Index: `IVF_FLAT` , COSINE, `nlist=128` (`:142-168`).
- **Sparse:** SPLADE++ (`naver/splade-cocondenser-selfdistil`) via pymilvus; it is the only sparse model accepted (`rag/model/sparse/base.py:8-38`). Index: `SPARSE_INVERTED_INDEX` , inner product.
- `<attachment>` tags are removed before embedding (`:109-113`). Both model names are stored in the index parameters, which is how the retriever later knows which models to load (`:258-279`).

4. Retrieve: query to ranked chunks

`Retriever` (`rag/retriever.py`) embeds the query with both models, sends one dense and one sparse ANN request (`nprobe=10`), and fuses them with Milvus `WeightedRanker(w, 1-w)` , where `w = hybrid_search_weight` (default 0.5) weights the dense side (`:151-259`). Optional filters: `document_ids` and `min_score` . The `k` fused hits are then reordered by a cross-encoder, `BAAI/bge-reranker-base` by default (`:304-345`). The default `k` is 1 (`:36`); the example config uses 5. `use_web: true` prepends DuckDuckGo snippets.

5. Answer: RAG

`RAGPipeline` (`rag/pipeline.py`) is a LangChain chain: validate input, retrieve, format the context as `[rank] text` blocks joined by blank lines (`:117-121`), fill a system prompt ("Use the following context… If none of the context answer the question, just say you don't know", `:28-33`), call the LLM, and return the answer plus the documents (`:124-179`). The LLM comes from OpenAI, Anthropic, Mistral or Cohere by model name, any OpenAI-compatible server through `base_url` (vLLM), or a local Hugging Face pipeline (`rag/llm.py:101-239`). Default temperature 0.7. `mmore rag` runs a JSONL of questions (`local`) or a FastAPI server (`api`) (`run_rag.py:215-330`).

6. Corrective judge (optional)

`retrieve_with_judge` (`rag/judge/corrective.py:76-180`) retrieves, then asks the judge. If configured score thresholds (similarity, rerank score, number of docs) all pass, it proceeds without calling an LLM (`rag/judge/evaluator.py:31-67`). Otherwise an LLM returns JSON with one of four decisions: PROCEED, ADD_QUESTIONS (up to 3 sub-queries), ADD_CONTEXT (web search) or RE_RETRIEVE (new query and/or larger k, default `max(2k, k+3)`). New hits are merged, de-duplicated and reranked against the original question. One corrective step by default.

7. Page-image retrieval: colvision

A separate three-command path (`colvision/`). `process` renders every PDF page at 200 dpi and embeds it with ColQwen2.5 (multi-vector, 128 dimensions per patch), saving vectors and the page's PyMuPDF text to Parquet (`colvision/run_process.py:126-158`). `index` stores every patch vector as its own Milvus row (FLAT, inner product). `retrieve` searches with each query-token vector, gathers candidate pages, then scores each with exact MaxSim (`colvision/milvuscolvision.py:173-252`). It returns the page's text layer as the document content (`colvision/retriever.py:150-181`) and is not wired into `RAGPipeline`.

8. Services and extras

- **Live indexing API** (`run_index_api.py`): upload, replace or delete files by `fileId`. It processes with marker, chunks in memory and rebinds chunk ids to `<fileId>+<i>` (`:72-81`), so provenance survives on this path. `mmore live-retrieval` serves it together with the retriever API.

- **Retriever API** (`run_retriever.py`): `POST /v1/retrieve` , `GET /v1/chunks/{fileId}/{chunkId}` , `GET /list_files` .
- **Privacy mode** (`privacy/` , `rag --privacy`): a LangGraph chain analyzer, detector, sanitizer, leakage adversary and human gate before the answer LLM. This is the previous thesis's contribution.
- **Web-search RAG** (`websearchRAG/`): sub-queries, DuckDuckGo or Tavily, summaries, then a merge with the RAG answer.
- Also: a RAGAS evaluator (`rag/evaluator.py`), a terminal UI (`tui/`), a profiler, and a new paper-discovery module.

Verified issues

Seventeen issues, most severe for the thesis first. "Run" means reproduced in the project's `.venv` (pymilvus 2.6.6, chonkie 0.5.1); "Read" means found by reading the code only.

#	Issue	Where	Evidence	Impact
1	Chunk ids are `hash(text)` , salted per process and not saved to JSONL	`type.py:90` , `:98-109`	Run: same text, two processes, two ids	`document_id` never links a chunk to its file in the CLI flow; re-indexing duplicates chunks. The live API rebinds ids and avoids this.
2	Page/paragraph positions lost when `postprocess` reads JSONL	`chunker/multimodal.py:254`	Run: positions present in memory, `None` after a JSONL round trip	No page citations in the standard CLI flow
3	`search_type` "dense" and "sparse" are swapped	`rag/retriever.py:59` , `:246-249`	Run: weight 0 ranked by sparse score, weight 1 by dense	Hybrid (0.5) is unaffected; any dense-vs-sparse comparison would be mislabelled
4	Reranker only reorders the k hits it is given	`rag/retriever.py:377-418`	Read	Raising k is the only way to give the reranker more candidates
5	Default PDF path (fast mode) does no OCR	`process/dispatcher.py:71` , `pdf_processor.py:264-332`	Read	Scanned PDFs yield little or no text unless `use_fast_processors: false`
6	`clean_text` merges paragraphs, squeezes spaces, rewrites emails	`process/utils.py:27-58`	Run	Fast-mode paragraph positions collapse to whole pages; spreadsheet columns lose alignment; email headers lose addresses

#	Issue	Where	Evidence	Impact
7	Last image of every document is dropped	`chunker/multimodal.py:77`	Run: 1 image in, 0 out; 3 in, 2 out	Single-figure documents lose their figure
8	Crawler drops or crashes on local paths when given 2+ URLs	`process/crawler.py:31-38`	Run: 1 dir + 2 URLs gives 0 files; 2 dirs + 2 URLs raises `KeyError`	Mixed web + local crawls fail
9	HF answers come back with the prompt; only a ChatML marker is stripped	`rag/llm.py:199-231`, `rag/pipeline.py:140`	Read (LangChain's `skip_prompt` defaults to False)	With a non-ChatML chat template the answer may contain the whole prompt
10	Marker path ignores `extract_images: false`	`pdf_processor.py:129-130`, `:199-200`	Read	Images are always extracted in single-GPU marker mode
11	Multi-GPU marker path: no pagination, and `None` added for failed files	`pdf_processor.py:359-366`, `:381`	Read	No page positions; one bad PDF can likely crash the save step
12	`PDFProcessor` keys in `processor_config` are never read	`pdf_processor.py:74-97`	Read (grep)	Marker batch sizes in the example config have no effect
13	Custom Whisper models ignored outside the device path	`media_processor.py:77`, `:124`, `:156`	Read (`staticmethod` called on an instance does not bind `self`)	`normal_model` / `fast_model` settings are silently skipped
14	`.aac` is crawled but no processor accepts it	`run_process.py:131`, `media_processor.py:65-74`	Read	Those files are skipped with a warning
15	Multimodal dense embedder pools `hidden_states[0]`	`rag/model/dense/multimodal.py:81`	Read	That layer is the token-embedding layer, before any transformer block: vectors carry no context
16	RAGAS evaluator: per-query partition ignored, one metric key garbled	`rag/evaluator.py:34`, `:177`	Read	Every eval query searches the whole collection
17	Distributed mode drops `device` and `fail_on_error`	`process/dispatcher.py:287-370`	Read	The new option only works for local runs

Where mmore stands against the state of the art

mmore covers the standard 2024 text RAG stack (hybrid dense + SPLADE search, cross-encoder reranking, a CRAG-style judge) and broader ingestion than most research systems. It is behind on four fronts that recent work treats as decisive: page-image retrieval feeding the generator, per-query adaptation, abstention, and multilingual retrieval. Each row compares against a paper from the thesis reading list.

Capability	State of the art	mmore today	Gap
Document parsing	OHRBench: no OCR tool tested was good enough to build a RAG knowledge base. OmniDocBench: pipeline tools and VLMs win on different document types	Marker OCR (opt-in) or the PyMuPDF text layer (default); 15+ formats including audio, video and email	Broad formats; default path has no OCR; no parsing-quality measurement
Visual document retrieval	ColPali and ViDoRe V3: visual retrievers beat textual ones on visually rich pages	ColQwen2.5 retrieval with exact MaxSim, as a separate path	Close on retrieval; not part of RAG
Generating from pages	VisRAG: 20-40% end-to-end gain over text RAG by generating from page images	Generator only ever receives text (colvision returns the text layer)	Missing
Hybrid fusion	Fixed hybrid + reranking recommended by Wang et al. 2024; DAT sets the dense/sparse weight per query	Fixed weighted sum, one global weight	Per-query weighting missing
Reranking	Rerank a large candidate set: top-1000 BM25 in Nogueira & Cho, k=100 in RetrievalRouter	Reranks only the final k hits	Reranker cannot add recall
Per-query routing (RQ1)	RetrievalRouter routes among 7 pipelines from the query; LTRR, R³AG route among retrievers; Adaptive-RAG by question complexity	One static configuration per run	Missing: this is the thesis's opening

Capability	State of the art	mmore today	Gap
Post-retrieval correction (RQ2)	CRAG: evaluator, then refine or web fallback; Beyond the Query: retrieval signals add no reliable gain over the query alone	CRAG-style LLM judge with 4 actions and threshold early exit	Comparable; not evaluated against a query-only control
Abstention	Sufficient Context: check whether the context suffices, abstain when it does not	Prompt says "say you don't know"; no abstain decision	Missing
Multilingual retrieval	BGE-M3: one multilingual model for dense, sparse and multi-vector; MIRAGE-Bench evaluates 18 languages	Defaults are English: all-MiniLM-L6-v2 and SPLADE++ trained on English MS MARCO; any HF dense model can be swapped in	Sparse side is English-only
Provenance	ViDoRe V3 annotates relevant pages and bounding boxes	Page/paragraph positions exist but are lost in the CLI flow; kept in the live API	Fixable bug
Evaluation	MIRAGE/MedRAG: 7,663 questions over 41 corpus, retriever and LLM combinations	Paper evaluates PubMedQA only; RAGAS wrapper ignores per-query partitions	No benchmark on humanitarian documents

Changes I would make

Fix the bugs that would corrupt a routing experiment first, then build the experiment harness, then add the capabilities each research question needs. You remain the author: each item below is a proposal with its reason, to discuss before coding.

Priority 1: fixes before any experiment

1. **Deterministic ids.** Derive chunk ids from `hashlib.sha1(file_path)` plus the chunk index, and serialize `id` and `document_id` in `to_dict`. Without this, per-query oracle labels cannot be joined across runs (issue 1).
2. **Keep page positions through JSONL.** Rebuild `PDFMetadata` in `from_dict` when `paragraph_starts` is present, or read positions from `metadata.extra` in the chunker (issue 2). Needed for page-level citations and for comparing text chunks with colvision pages.
3. **Swap the search weights back** to `{"dense": 1, "sparse": 0}` (issue 3). Dense-only and sparse-only are two of the configurations a router would choose between.
4. **Separate `k_retrieve` from `k_final`.** Retrieve e.g. 50 and rerank down to k (issue 4), as both Nogueira & Cho and RetrievalRouter do. This also makes "rerank or not" a meaningful routing decision, as in Adaptive Re-Ranking.
5. Smaller fixes, each one line: the image off-by-one (issue 7), `extract_images` in the marker path (10), preserving `\n\n` in `clean_text` (6), and the crawler key loop (8).

Each fix is a natural first PR: reproduce, test, fix, as you planned.

Priority 2: an experiment harness

- **A configuration grid** run over one question set: {fast, marker} parsing x {dense, sparse, hybrid} x {rerank on, off} x k, plus colvision. Log per-query retrieval and answer scores to build the oracle labels (best configuration per query).
- **Deterministic generation** (temperature 0) for every run, and a fixed seed for any LLM judge.
- **A humanitarian question set** with page-level gold evidence, including scanned and French documents. Nothing public covers this setting; it is the thesis's main data contribution.
- **Significance testing** (paired randomization or bootstrap, per Smucker et al. 2007) when comparing configurations and the router against the oracle.

Priority 3: research-question features

- **RQ1, query-only router.** A small classifier (start with XGBoost, as LTRR found best) over query features: the pre-retrieval QPP features of Hauff et al. and the external features of LLM-Independent Adaptive RAG. It picks a configuration from the grid. Baselines: the best static configuration, Adaptive-RAG, and the oracle.

- **RQ2, post-retrieval decisions.** Add an ABSTAIN decision to the judge, driven by a sufficiency check as in Sufficient Context. Replace the web fallback (ADD_CONTEXT) with "rerun with another configuration", for example colvision. Always compare against a matched query-only router, the control Beyond the Query recommends.
- **RQ3, page images in RAG.** Wire `ColVisionRetriever` into `RAGPipeline` as a selectable retriever. Pass page images, not the text layer, to a vision-language generator (VisRAG). Offer the choice text vs page as a routing arm, as in RetrievalRouter.
- **Multilingual defaults.** Evaluate BGE-M3 (dense + sparse in one multilingual model) against MiniLM + SPLADE on French documents before choosing defaults.

What I would leave alone

The privacy layer, web-search RAG, paper discovery and the TUI are outside the three research questions. The NER and translation post-processors are only worth touching if translation becomes a routing arm.

Sources

- Code: the mmore repository at commit `1377ce0` (Oct 2026), read file by file; experiments run in its `.venv`.
- MMORE paper (Sallinen et al., 2025)
- Papers compared against: OHRBench, OmniDocBench, ColPali, ViDoRe V3, VisRAG, Wang et al. 2024, DAT, Nogueira & Cho, RetrievalRouter, LTRR, R³AG, Adaptive-RAG, CRAG, Beyond the Query, Sufficient Context, BGE-M3, MIRAGE-Bench, MIRAGE/MedRAG
- The thesis reading list, for the remaining papers named in the proposals (Hauff et al. 2008, LLM-Independent Adaptive RAG, Adaptive Re-Ranking, Smucker et al. 2007)